Visually-Grounded Library of Behaviors for Generalizing Manipulation Across Objects, Configurations and Views

Hsiao-Yu Tung*, Jingyun Yang*, Yunchu Zhang*, Gaurav Pathak, Ashwini Pokle,
Christopher G. Atkeson, Katerina Fragkiadaki

***Abstract*— We propose a visually-grounded library of behaviors approach for manipulating diverse objects across varying initial and goal configurations and camera placements. Our key innovation is to disentangle the standard image-to-action mapping into two separate modules that use different types of perceptual input: (1) a behavior selector which conditions on the *static* object properties to select the behaviors that can successfully perform the desired tasks on the object in hand, and (2) a library of behaviors each of which conditions on the *dynamic* object properties to predict actions to execute over time. The selector uses a semantically-rich 3D object feature representation extracted from images using geometry-aware 2D-to-3D neural networks. This representation is trained to be view-invariant and affordance-aware in a self-supervised manner. The library consists of different types of behaviors that condition on abstract object representations that capture object dynamics (such as 3D object centroids) and map them to robot actions. We test our framework on simulated grasping and pushing of diverse objects as well as grasping in a real robot setup. Our model generalizes better than image-to-action mappings that do not factorize static and dynamic object properties, state-to-action mappings that do not consider contextual visual features, and library-based approaches whose selectors do not consider view-invariant, affordance-aware 3D feature representations.**

I. Introduction

When robots make their way out from factories into people's houses, they are expected to be able to manipulate a diverse set of objects that might appear in a house. Such a goal is challenging since it is hard to predict what a robot might encounter during execution. Possibly, the robot would need to grasp previously unseen objects placed in arbitrary locations, and it might be difficult to observe this object in its most familiar views due to obstacles that might block certain viewpoints. In this paper, we explore how we can best **incorporate visual information** to learn robot behaviors that generalize to novel objects, configurations and views.

Despite advances, current policy learning research focuses on discovering new behaviors and generalizing to new types of tasks as opposed to improving model generalization on unseen objects and camera views. Existing research in reinforcement learning mostly considers the same objects at training and test time, and state representations that abstract away object appearance information, such as color and texture that might help identify the material the object is made out of or coated with and thus provide information about

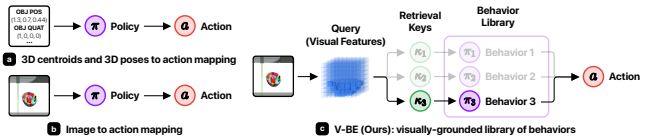


Fig. 1: We propose a novel policy representation that improves model generalization on unseen objects and camera views. In contrast to prevalent approach of learning a *state-to-action* or *image-to-action* mapping, the proposed model decomposes a policy into a behavior selection module and a library of behaviors to select from. This decomposition enables different state representations to be used for each behavior and the selection module.

appropriate friction or contact models. To handle objects with varying appearance, existing visuomotor approaches incorporate visual information by learning a direct mapping from images to the target actions that will lead to successful task execution. Previous work leveraging direct image-to-action mappings has shown successful task execution in the domain of grasping [22] [16], flying [7] and pushing [14], [1]. The ability to robustly generalize across unseen camera viewpoints and objects has not been convincingly demonstrated.

Lack of generalization in existing visuomotor approaches is attributed to: (a) lack of state abstraction and (b) occlusions, foreshortening, and other artifacts caused by camera projection in images. To generalize a policy to a new object, It is often useful to find a state abstraction, a minimal subset of environment features useful for performing a task and are shared across objects [11]. Abstractions facilitate transfer by explicitly ignoring parts of the environment that are irrelevant to a particular skill. For example, when opening a bottle, the size of the cap is a relevant feature, but the color of the bottle itself is irrelevant. Policies that attempt to learn what features matter automatically often require large amounts of training examples. Aside from a lack of state abstraction, existing visuomotor policies are sensitive to changes in the pixel values or image space. Appearance and location of the object and the gripper and their relative arrangement, which may be constant in 3D, varies with the viewpoint in 2D images, which causes these policies to fail under camera viewpoint changes.

To this end, we propose Visually-grounded library of BEhaviors (V-BEs), a novel manipulation policy representation that can handle manipulation tasks with varying objects, initial and goal configurations, and viewpoints. In place of prevalent approaches of learning a *flat* image-to-action or

*Equal contribution.

The authors are with Carnegie Mellon University, Pittsburgh, PA 15213. Emails: {htung,jingyuny,yunchuz,gauravp,apokle,cga,kfragki2}@andrew.cmu.edu

object-to-action mapping (see Figure 1(a)(b)) which operates on a single representation, our model learns to disentangle and operate on two separate representations in a *hierarchical* setup (see Figure 1(c)). At the higher level of the hierarchy, a selector selects an appropriate behavior to execute among a library of given behaviors at the lower level of the hierarchy. This selector uses a rich, affordance-grounded, and view-invariant representation that captures the **static** object properties such as object shape and interactions that can happen with the object. The lower level of the hierarchy consists of a library of distinct behaviors, each mapping a given state to an action. Each behavior uses an abstract state representation that captures **dynamic** properties of entities in the environment such as object position and gripper position at the current time step. Different behaviors can be redundant, use the different state representations, and be learned or manually engineered with different algorithms [15], [2], [5]. At inference time, the selector receives a rich visual observation from the environment at the start of a task and outputs a selected behavior in the library of behaviors to be executed. The selected behavior then takes the abstract object and gripper positions at each time step as input and makes interaction with the environment to execute the task.

To reliably select appropriate behaviors while handling changing camera views, we propose to use an affordance-aware, viewpoint-invariant 3D object feature representation in the selector. This representation is learned via a combination of self-supervised view-prediction and object interaction prediction. The former makes the representation robust to changing camera views, while the latter helps the representation encode affordance information (i.e., how objects change by applying a behavior).

We test our representations using simulated grasping and pushing with a large pool of diverse objects. We show that the proposed representation can grasp and push diverse objects with varying object starting positions, initial poses, goals, and camera views. It outperforms existing single-policy image-to-action mapping or object-to-action mapping that uses only the fast changing 3D object locations [20]. We further show our model trained in simulators could be directly transferred to a real robotic platform without fine-tuning, and can significantly improve its performance by fine-tuning only on a handful of interaction labels. In the real robot setup, the robot can use camera views that are totally different from those used at training time in the simulator to get good performance.

II. Related Work

Libraries of Behaviors: Libraries of behaviors have been considered in many previous works, and library elements have been called action, motion, or motor primitives, parameterized policies, options, chunks, macros, subroutines, behavioral units, and skills, as well as many other terms. Most previous work does not use perception, but rather assumes the environment state is known, or has a different approach to perception than ours. Work of Neumann et al. on modular policies [18] defines a motor primitive as a mapping from a robot's joint angles and velocities and not the locations of objects. Rather, object locations are used as context for selecting a motor primitive. This means i) motor primitives are "blind" to the object 3D location throughout the episode, and ii) object appearance is discarded which means the library of primitives cannot handle object variability. Work of de Silva [31] on parameterized policies similarly considers as the contextual vector the desired goal location of a dart, and learns a mapping from this location to policy parameters, where each policy operates only over the robots' joints and is represented as a Dynamic Motion Primitive (DMP) [29], [28], [34]. To extend DMP to tasks involving physical interactions, Kroemer et al. [12] parameterize DMPs based on the distance of an object part from the gripper, but only consider known object 3D shapes. Strudel et al. [33] use a depth frame as input to a meta-policy that selects skills to execute. However, the use of modularity over skills is for temporal sequencing: there is no mixture of behaviors per manipulation task, rather an image to action mapping is learnt with behavior cloning from demonstrations.

Deep Reinforcement and Imitation Learning for Object Manipulation: In current work in (deep) reinforcement and imitation learning, typically one policy is learned via reinforcement learning [2], [10], [38], imitation learning [25], [39], [30], or a combination of the two [24], [4]. Deep RL algorithms are commonly trained and evaluated in a fixed environment in terms of their ability to discover a policy rather than their ability to generalizing it to previously unseen circumstances. Almost all of this literature parameterizes the learnt policies in terms of object and gripper 3D centroids and their velocities, and ignores contextual visual information [19]. Several works have suggested that image-based policies can be discovered as well but more slowly [26]. To facilitate learning of the image-to-action mapping, [21] uses state information (object centroids) for the critic network to guide the learning of an image-based policy network. There is also works that distill image-based policy by imitating trajectories obtained from a planner operating in a low-dimensional state space [14], [36]. These methods have not shown so far to generalize well across objects and views. Florence et al. [6] train manipulation policies parameterized by the 2D or 3D locations of a designated set of visual descriptors. They pretrain their visual feature space using correspondences from triangulation and augmentation of images by resizing and warping. While [6] shows their policies generalize across objects of the same object category, e.g., shoes of different shapes, our policy representation further allows generalization across object categories, and no explicit descriptor selection or optimization is necessary.

Hierarchical Reinforcement Learning: Our method is closely related to hierarchical reinforcement learning (HRL) algorithms. While most existing HRL approaches [32], [13] focus on discovering behaviors and temporally sequencing them in a complex task, this paper aims to use hierarchical structures to improve model generalization across objects, configurations, and views. This is a critical yet challenging problem that has not been addressed in the HRL literature.

III. METHOD

Our policy Π is a visually-grounded library of behaviors, as shown in Figure 2. V-BEs are comprised of multiple behaviors $\{\pi_i \mid i = 1, 2, \ldots, K\}$ and a behavior selector G. Each behavior π_i is either an open loop trajectory generator, or a closed-loop policy (feedback controller) that tracks objects over time. Since most behaviors are developed using abstract state representations, namely, 3D object locations and 3D poses and do not take into account object shape or appearance, each of them can handle only a subset of objects and their 3D orientations. To be able to handle a diverse range of objects, we need to decide which behavior to use under which circumstance. To achieve this, we propose a behavior selector which is a gating network that, given the sensory observations $\emptyset_t$, predicts the behavior that can best manipulate the object under consideration to its desired configuration. Our sensory input to action mapping then, at each time-step reads:

$$\Pi(a_t|o_t, g) = \sum_{i=1}^{K} G(o_t)_i \pi_i(a_t|o_t, g), \tag{1}$$

where g is the goal configuration to achieve, $o_t = \{\mathbf{I_{v_t}}, \mathbf{v}_t\}$ is the sensory observation in the form of one or more RGB-D images $\mathbf{I_v} \in \mathbb{R}^{\hat{H} \times \hat{W} \times 4}$ captured from arbitrary but known camera poses $\mathbf{v} \in \mathbb{R}^7$, and $G(o_t)_i \in \{0, 1\}$ denotes the selector's output for π_i at time step t.

Our main contribution is the factorization of the visual properties of the object presented in the sensory observation o_t, so that, the behaviors $\{\pi_i \mid i = 1, 2, \ldots, K\}$ and the selector G operate on separate object visual properties. The behaviors use as input *dynamic* visual properties that change throughout the manipulation episode, such as object or part 3D positions, and abstract away object appearance information. Let $f^{\pi_i}(o_t)$ denote the state abstraction behavior i uses. In contrast, the selector G takes as input the *static* object visual properties. It takes the raw sensory observation o_t and transforms it into a location invariant visual representation of the object $f^G(o_t)$, which captures object appearance and 3D size, which remain constant throughout the manipulation. With such a distinction on the representation used by the two modules, we can rewrite Equation (1) into:

$$\Pi(a_t|o_t, g) = \sum_{i=1}^{K} G(f^G(o_t))_i \pi_i(a_t|f^{\pi_i}(o_t), g). \tag{2}$$

One advantage of having this decomposition is that each behavior π_i can then use its own state abstraction $f^{\pi_i}(o_t)$, such as object 6D-poses, axis-aligned bounding boxes (as used in this paper), part-based 3D boxes, or object keypoints locations [23]. This framework supports integrating a wide range of existing models, representations, and behaviors as selectable behaviors. This architecture is also recursive, in that any behavior can consist of the same architecture replicated at a lower level. We consider here single object manipulation skills but our formulation can generalize to multiple objects.

In the rest of this section, we first explain the architecture and training details of the behavior selector in Section III-A, and then explain how we acquire a library of behaviors for the two robot manipulation tasks we evaluate our framework on in Section III-B.

A. Visually-Grounded Behavior Selector

The behavior selector G is a nearest neighbor classifier that uses the visual feature representation of the object to manipulate $f^G(o_t)$ as a query to retrieve the behavior that has the highest propability to manipulate the object successfully. Each behavior π_i is associated with a learned retrieval key κ_i.

View-invariant Feature Learning Self-supervised by View prediction: The behaviour selector should ideally be viewpoint invariant as behaviors should not depend on camera placement in the environment in most cases. To achieve this, we design the selector G to operate on a 3D viewpoint-invariant object-centric feature representation $f^G(o_t) = \mathbf{M}^o \in \mathcal{M} = \mathbb{R}^{H \times W \times D \times C}$, which is a set of 3-dimensional feature maps centered around the object of interest. The retrieval key $\kappa \in \mathcal{M}$ is also learned in the same representation space. Both $\mathbf{M}^o$ and κ are with the size of $64 \times 64 \times 64 \times 32$ in the experiments. We build upon recent advances in 3D perception and use geometry-aware networks (GRNNs) [37] to map a single RGB-D image or a set of multi-view images to a complete 3D feature representation of the scene the image(s) depict. GRNNs learn to complete the missing information from a single view by optimizing end-to-end for view prediction using multi-view data collected by the robot [37]: an input RGB-D image is un-projected into an (incomplete) 3D feature grid, missing information is "inpainted" via a series of 3D convolutions, and the completed 3D feature grid is projected to a novel viewpoint and decoded to the corresponding RGB-D image. The view prediction loss reads:

$$\mathcal{L}_{\text{view-pred}}(\phi) = \sum_{n=1}^{N} \| P_\phi(\text{GRNN}_\phi(\mathbf{I}^n_{\mathbf{v}^n}, \mathbf{v}^n), q) - I^n_q \|_2, \tag{3}$$

where $\text{GRNN}(\mathbf{I_v}, \mathbf{v})$ is a function that lifts RGB-D input images $\mathbf{I_v}$ capturing from camera poses $\mathbf{v}$ to a geometry-consistent 3D feature map $\mathbf{M} = \mathbb{R}^{\bar{W} \times \bar{H} \times \bar{D} \times \bar{C}}$ with $\bar{W}, \bar{H}, \bar{D}$ denoting the spatial dimension and $\bar{C}$ denotes the feature dimension of the 3D feature map, $P(\mathbf{M}, q)$ is a projection function that projects the 3D feature map from a query viewpoint q and decodes to a target image, I^n_q is the target image to predict, and ϕ is the neural network weights of GRNN. From the scene map $\mathbf{M}$, we obtain object-centric feature representation $\mathbf{M}^o$ by cropping the scene map using a fixed-size axis-aligned box, centered around the object we wish to manipulate.

Our selector G, given a 3D object feature representation $\mathbf{M}^o$ of the object to be manipulated and a retrieval key κ_i for behavior π_i, computes the probability $\hat{G}_i$ that π_i can successfully manipulate the object to its desired goal location:

$$\hat{G}(\mathbf{M}^o)_i = \sigma(\langle \mathbf{M}^o, \kappa_i \rangle) \in [0, 1], \tag{4}$$

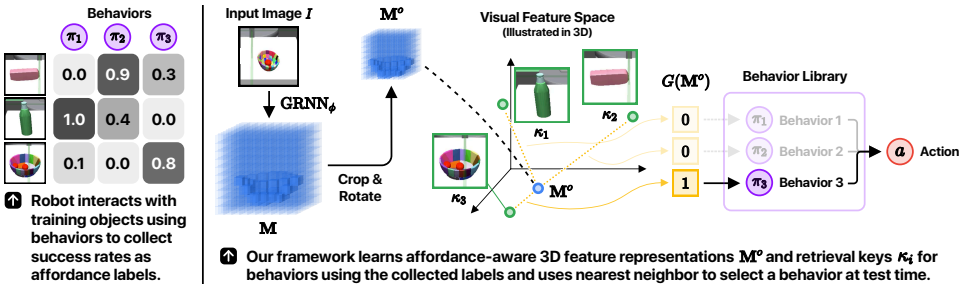


Fig. 2: Overview of the proposed framework.

where $\langle \cdot, \cdot \rangle$ is the inner product operation and σ is the sigmoid function. It then selects the behavior with the highest predicted probability:

$$G(f^G(o_t))_i = G(\mathbf{M}^o)_i = \mathbb{1}\{i = \text{argmax}_{i'} \hat{G}(\mathbf{M}^o)_{i'}\}. \quad (5)$$

Affordance-aware Feature Learning Self-supervised by Interaction: The 3D feature representations obtained through view prediction capture how objects look, but not how they can be manipulated. Our selector should operate over representations that capture information about how an object can be used, and how it will react to an agent's actions. We will use the term *affordance*, coined by Gibson in 1966 [8], to denote this information.

We learn affordance-aware 3D feature representations by applying behaviors in the library to the objects in the training set, and update the object representations and the retrieval keys $\kappa_i, i = 1 \cdots K$ for behaviors $\pi_i, i = 1 \cdots K$ according to the success rate a behavior achieves on a particular training object, as shown in Figure 2. Let $u_{ij} \in [0, 1]$ be the success rate after N_{attempt} trials of behavior π_i on object ω_j, under potentially varying goals g. Here, we treat same objects with different initial orientation as different objects since we assume that canonical poses of objects are not given. Each trial encounters the same object in a potentially different initial location, orientation and desired goal location. Our loss then reads:

$$\mathcal{L}_{\text{affordance}}(\kappa_1 \cdots \kappa_K, \phi) = \sum_{i=1}^{K} \sum_{j=1}^{|\Omega_{\text{train}}|} \sum_{n=1}^{N_{\text{attempt}}} \text{BCE}\big(\mathbb{1}\{u_{ij} \geq \delta\}, \langle R_\phi(I_n^{\omega_j}), \kappa_i \rangle\big), \quad (6)$$

where $\text{BCE}(p, l) = -p \cdot \log(\sigma(l)) + (1 - p) \cdot \log(1 - \sigma(l))$ is the standard binary cross entropy loss, δ is a hyperparameter for thresholding the success rates, $I_n^{\omega_j}$ are the initial RGB-D images drawn from tasks with object ω_j in the nth trial, $R(I)$ is the 2D-to-3D visual feature extractor that takes the cropped object-centric 3D feature representations from GRNNs, and ϕ is the GRNNs weights. The final objective for training the affordance-based visual features is

$$\underset{\kappa, \phi}{\text{minimize}}\ \mathcal{L}(\kappa, \phi) = \mathcal{L}_{\text{view-pred}}(\phi) + \lambda_{\text{a}} \cdot \mathcal{L}_{\text{affordance}}(\kappa, \phi), \quad (7)$$

where λ_{a} is a hyperparameter for balancing the two losses.

B. Building a Library of Behaviors

The visually-grounded behavior selector selects plausible behaviors to manipulate a specific object from a library of behaviors. But how can we obtain this library in the first place? Indeed, any existing behaviors, whether engineered or learned using reinforcement learning or imitation learning, can be included in our library. In this paper, we consider two common manipulation tasks: pushing and grasping, and build appropriate behavioral libraries for each.

For pushing, the behaviors are deterministic goal conditioned policies $a_t = \pi(s_t, g), t = 1 \cdots T$ that map a state of the environment and the robot $s_t = [s_t^e, s_t^r]$ to an action at every time step. In our case, the state of the environment is the 3D object centroid $s_t^e = f(o_t, O)$ and the robot state (gripper 3D location, pose, and whether it is opened or closed). The action includes 3D motion, opening (position control), and closing (force control) of the gripper. We use a total of 25 goal conditioned policies – one is trained from the whole set of objects, while the others are trained on disjoint subsets of object configurations organized based on object category and initial poses. We train all policies using deterministic policy gradients (DDPG) [24] with goal relabelling (HER) [2] while randomizing initial and goal object 3D locations.

For grasping, we design controllers $\pi(a_t | g; p^{grasp}, q^{grasp})$ which given a 3D grasping point $p^{grasp} \in \mathbb{R}^3$ relative to the center of the object and a grasping 3D angle $q^{grasp} \in \mathbb{R}^2$, move the gripper (open loop) to the grasping 3D point location, close it, and move it to the desired goal location. The grasping angle q^{grasp} consists of two numbers describing the yaw of the gripper and the elevation angles between the gripper and the table surface. When the elevation angle is smaller than 90 degrees (not top-down grasps), we constraint the gripper to point toward the center of the object on the x-y plane. We manually select 30 different controllers including top-down grasps with different yaw orientations (top-grasps) and grasps from the side with different elevation angles of the gripper (side-grasps). We empirically found that these parameterized controllers are quite stable and can be shared across multiple objects.

IV. Experiments

Our experiments aim to answer the following questions: (1) Does the proposed library-based approach outperform existing methods that use a single combined perception and policy module, either using 2D images, 3D object locations, or 3D scene feature maps as input? (2) Is the proposed view-invariant and affordance-aware 3D feature representation a necessary choice for the selector? (3) How do V-BEs com-

pare to state-of-the-art methods, specifically, in the grasping domain? (4) Does the method work on a real robot (rather than simulation)? We test our model on grasping and pushing a wide variety of objects in the MuJoCo simulator [35] and further test the grasping module on a real-world Franka Panda robot arm.

A. Simulation Experiment Setups

Our simulated environment consists of a Fetch Robot equipped with a parallel-jaw gripper. The robot is positioned in front of a table of height $0.4m$. To obtain the visual observations, on each episode we choose 3 random cameras from cameras placed at 30 nominal different views including 10 different azimuths ranging from $0°$ to $360°$ combined with 3 different elevation angles from $20°$, $40°$, $60°$. All cameras are looking at the center of the table top, and are 0.5 meter away from that point. All images have size 128×128.

Task Descriptions: In the grasping task, the agent has to grasp an object and move it to a specified target location above the table. We use 274 distinct object meshes from 6 categories in ShapeNet [3] including toy buses, toy cars, cans, bowls, plates, and bottles. We randomly split the dataset into 207 objects for training, and 67 objects for testing. After augmenting the mesh with random scaling from 0.8 to 1.5, and random rotations around the vertical (z) axis, we get a total of 800 distinct object configurations (object instance and pose), 600 for training and 200 for testing. At the beginning of each episode, an object is placed in an area of $30cm \times 16cm$ around the center of the table, and a goal is sampled uniformly around the gripper's initial position $10 \sim 30cm$ away from the gripper. An episode is regarded as successful if the object centroid is within $5cm$ of the goal position at the final timestep.

In the pushing task, the agent has to push an object placed on the table to a specified target location. We use 100 objects from 12 categories in ShapeNet [3]: baskets, bowls, bottles, toy buses, cameras, cans, caps, toy cars, earphones, keyboards, knives, and mugs. After augmentation and splitting to train and test sets, we obtain 615 training object configurations and 200 for testing. The initial and the goal position of the object are both uniformly sampled to be within $15cm$ of the center of the table along both x-axis and y-axis, although we resample if that location is already in the goal area. An episode is regarded as successful if the gripper pushes the object within $5cm$ of the goal position within 50 timesteps.

Baselines: We compare our method with various learning and non-learning based methods for object manipulation:

(a) *Single Behavior w/ Abstract 3D State (Abstract 3D) [2], [24]:* a policy takes as input ground truth 3D bounding box of the object and gripper and outputs actions.

(b) *Single Behavior w/ Abstract 3D State and 2D Images (Abstract 3D + Image):* a policy takes both RGB-D images and the ground truth 3D bounding box as inputs and outputs actions.

(c) *Single Behavior w/ 3D Feature Tensor (Contextual 3D):* a policy that takes as an input RGB-D images and the ground truth 3D bounding box and outputs actions. Different from (b), the model first transforms the image into a view-invariant 3D feature tensor using GRNNs [37], then converts the 3D feature tensor into a feature vector though three 3D-convolutional layers and a fully connected layer, and finally concatenates it with the rest of the inputs to predict actions.

(d) *Ours, Library of Behaviors w/ Visual Selector (V-BEs):* Our model takes the same input as (b) and (c). The 3D bounding boxes are used as input to all the behaviors. The RGB-D images are transformed into 3D affordance-aware visual features and treated as input to the selector.

All policies have a similar number of parameters. We train the baselines with different learning methods including behavior cloning [14], DDPG-HER [15], [2] and DAGGER [27]. We report the best performance we got by training the model with these different methods. For pushing, we found that using DDPG-HER is enough to lean a good *Abstract 3D* policy from scratch. For *abstract 3D + Image*, we found it is critical to use behavior cloning from expert demonstrations to obtain good policies. The expert demonstrations are obtained from trained expert policies on single objects. We follow the architectural design and training of [14] to learn the 2D-to-action mapping. For *Contextual 3D*, we include DAGGER to enforce behavior cloning during execution. To train the grasping policies, we further include human demonstrations in the replay buffer when training it with DDPG-HER. Both *abstract 3D + Image* and *Contextual 3D* are trained with DAGGER since offline behavior cloning is insufficient.

B. Single behavior versus a library of behaviors

We compare the proposed model with models that do not use a library-based approach, i.e., single behavior approaches. As shown in Table I, our method outperforms all the single behavior baselines. *Abstract 3D* performs well, but since it does not use any visual information, its performance saturates at around 0.8 for pushing and 0.3 for grasping. *Abstract 3D* performs poorly for grasping. The learned behaviors do not transfer well to new objects. Adding a 2D image helps, but not dramatically (see *Abstract 3D + Image* in Table I). Although 3D feature maps obtained from GRNNs are semantically rich and can handle varying viewpoints, the mapping to actions is harder to learn due to the higher dimensionality of the 3D scene map, resulting in underfitting models. Our model takes advantage of both abstract and semantically rich representation and thus can handle better object variability and transferability. The combinatorial nature of the proposed method allows the model to capture the multi-modality in trajectory generation.

	Single Behavior			library of behaviors
	Abstract 3D [2], [24]	Abstract 3D + Image	Contextual 3D	V-BEs (Ours)
grasping	0.30	0.35	0.20	**0.78**
pushing	0.83	0.70	0.10	**0.88**

TABLE I: Success rates on grasping and pushing unseen objects.

C. The necessity of building the selector with the proposed 3D view-invariant visual representations

Next, we show the importance of learning visual affordance features with a 3D visual feature representation and the importance of self-supervised training with interaction labels. We compare our method with two baselines: (a) a model with a selector that operates over 3D visual feature representation learned only with the view and occupancy prediction loss, as suggested in [37][9], without fine tuning with interaction labels, and (b) learning the visual affordance features using 2D visual features extracted from 2D CNNs. Our method significantly outperforms these two baselines, which shows the importance of fine-tuning on the execution labels as well as the use of 3D feature representations for behaviour selection.

	2D Features	Proposed Method (3D Features)	Proposed Method w/o Fine Tuning on Interatcion Labels [37]
Grasping	0.46	**0.78**	0.31
Pushing	0.81	**0.88**	0.46

TABLE II: Success rates on grasping and pushing unseen objects using selector with varying representations.

D. Comparison with other grasping baselines

We further compare the proposed model with current state-of-art methods in grasping: (a) **DexNet [16]:** a state-of-the-art top-grasps method which, given a top-down depth map, generates grasps as the planar position, angle, and depth of a gripper relative to an RGB-D sensor. (b) **6-DOF GraspNet (GraspNet) [17]:** a state-of-the-art grasping method that describes the grasp as a full 6-degrees-of-freedom (DoF) pose. For (a)(b), we use the publicly released code and model provided by the authors, finetune it with interactions in our training environments, and evaluate it in our test environments. (c) **Our Model with Only Top-grasps:** an ablated baseline using the proposed methods and include only the top-grasps. (d) **Our Model with object detector:** we further evaluate how our model performs with estimated object bounding box. To remove the effect of the detector, all other baselines use ground truth object locations to obtain the corresponding depth image or inputs.

As shown in Table III, our method outperforms all baselines by a large margin, even with an estimated object bounding box. *Dexnet* performs the best among the baselines, but it can only handle height maps (depth maps where depth is the vertical direction defined by gravity) while all other methods can use any camera view. *GraspNet*, to our surprise, performs poorly, especially when we randomize object initial and goal locations. Many grasps proposed by *GraspNet* often turn out to be unstable when the object is placed close or far away from the robot. Our model handles variances in object locations well because we include in the behavior library general top-down, left-hand, and right-hand grasping behaviors that are more stable across object locations. Our approach outperforms these methods by a large margin even only using very naive controllers in the behavior library.

DexNet [16]	GraspNet [17]	V-BEs w/ Top-Grasps	V-BEs (Ours)	V-BEs w/ detector
0.72	0.36	0.67	**0.78**	0.73

TABLE III: Success rates on grasping for unseen objects.

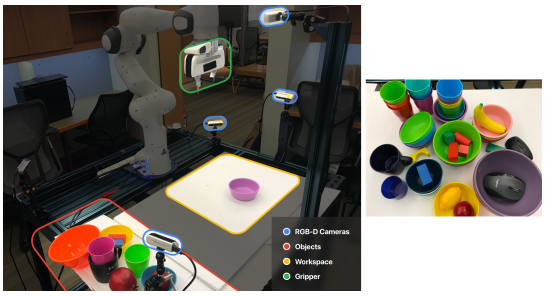


Fig. 3: Real robot setup (left) and objects used (right).

E. Real robot results

To demonstrate that our V-BEs representation works in a real robot setup, we measure the performance of our model on a real 7-DOF Franka robot arm equipped with a parallel-jaw gripper for the grasping task (see Figure 3). We set up 4 realsense RGB-D cameras that have full view of the workspace around the the center of the table. In each trial, an object is placed in a 20cm $\times$ 10cm region on the table, and a goal is sampled uniformly away from the objects start position from $-$20cm to 20cm in all x, y, and z dimensions. An episode is considered successful if the object centroid is within $5cm$ of the goal position.

We use the same set of grasping behaviors as in our simulated experiments. We randomly select 3 views from 4 possible camera views to obtain RGB-D images as inputs to the learned selector. We randomly split 56 objects including computer-mouses, bananas, bowls, mugs and wooden bricks (see Figure 3) into a training set and a test set, each with 28 objects. Without further fine-tuning, our model trained in simulated environments achieves success rate of 46.4% on the test set, even though the camera views and objects are unseen at training time. After fine-tuning with interaction labels collected on the training objects in the real robot setup, our model can achieve success rate of 82.1% on the test set. Qualitative results are included in our supplementary video.

V. Conclusion

We have presented a novel representation, V-BEs , for manipulation that can generalize manipulation skills across objects and views. Our key insight is to use abstract and dynamic properties that change during manipulation to parameterize the behaviors, and use viewpoint-invariant visual representations as context to select the appropriate behavior for each object and pose. Our proposed policy representation outperforms alternatives that use entangled visual representation and do not factorize state and visual context features in this way. Extending the proposed representation towards model-based control by sequencing the behaviors over time to discover longer duration policies and learning diverse state representations for multi-object or part based interactions are clear avenues for future work.

REFERENCES

[1] P. Agrawal, A. Nair, P. Abbeel, J. Malik, and S. Levine, "Learning to poke by poking: Experiential learning of intuitive physics," *CoRR*, vol. abs/1606.07419, 2016. [Online]. Available: http://arxiv.org/abs/1606.07419

[2] M. Andrychowicz, F. Wolski, A. Ray, J. Schneider, R. Fong, P. Welinder, B. McGrew, J. Tobin, P. Abbeel, and W. Zaremba, "Hindsight experience replay," *CoRR*, vol. abs/1707.01495, 2017. [Online]. Available: http://arxiv.org/abs/1707.01495

[3] A. X. Chang, T. Funkhouser, L. Guibas, P. Hanrahan, Q. Huang, Z. Li, S. Savarese, M. Savva, S. Song, H. Su, J. Xiao, L. Yi, and F. Yu, "ShapeNet: An Information-Rich 3D Model Repository," Stanford University — Princeton University — Toyota Technological Institute at Chicago, Tech. Rep. arXiv:1512.03012 [cs.GR], 2015.

[4] L. Fan, Y. Zhu, J. Zhu, Z. Liu, O. Zeng, A. Gupta, J. Creus-Costa, S. Savarese, and L. Fei-Fei, "Surreal: Open-source reinforcement learning framework and robot manipulation benchmark," in *Proceedings of The 2nd Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, A. Billard, A. Dragan, J. Peters, and J. Morimoto, Eds., vol. 87. PMLR, 29–31 Oct 2018, pp. 767–782. [Online]. Available: http://proceedings.mlr.press/v87/fan18a.html

[5] S. Feng, X. Xinjilefu, C. G. Atkeson, and J. Kim, "Optimization based controller design and implementation for the atlas robot in the darpa robotics challenge finals," in *2015 IEEE-RAS 15th International Conference on Humanoid Robots (Humanoids)*, 2015, pp. 1028–1035.

[6] P. Florence, L. Manuelli, and R. Tedrake, "Self-supervised correspondence in visuomotor policy learning," 2019.

[7] D. Gandhi, L. Pinto, and A. Gupta, "Learning to fly by crashing," *CoRR*, vol. abs/1704.05588, 2017. [Online]. Available: http://arxiv.org/abs/1704.05588

[8] J. J. Gibson, *The Ecological Approach to Visual Perception*. Houghton Mifflin, 1979.

[9] A. W. Harley, F. Li, S. K. Lakshmikanth, X. Zhou, H. F. Tung, and K. Fragkiadaki, "Embodied view-contrastive 3d feature learning," *CoRR*, vol. abs/1906.03764, 2019. [Online]. Available: http://arxiv.org/abs/1906.03764

[10] O. Kilinc, Y. Hu, and G. Montana, "Reinforcement learning for robotic manipulation using simulated locomotion demonstrations," 2019.

[11] O. Kroemer, S. Niekum, and G. Konidaris, "A review of robot learning for manipulation: Challenges, representations, and algorithms," 2019.

[12] O. Kroemer and G. S. Sukhatme, "Learning relevant features for manipulation skills using meta-level priors," *CoRR*, vol. abs/1605.04439, 2016. [Online]. Available: http://arxiv.org/abs/1605.04439

[13] T. D. Kulkarni, K. Narasimhan, A. Saeedi, and J. B. Tenenbaum, "Hierarchical deep reinforcement learning: Integrating temporal abstraction and intrinsic motivation," *CoRR*, vol. abs/1604.06057, 2016. [Online]. Available: http://arxiv.org/abs/1604.06057

[14] S. Levine, C. Finn, T. Darrell, and P. Abbeel, "End-to-end training of deep visuomotor policies," *CoRR*, vol. abs/1504.00702, 2015. [Online]. Available: http://arxiv.org/abs/1504.00702

[15] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. M. O. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," *CoRR*, vol. abs/1509.02971, 2016.

[16] J. Mahler, J. Liang, S. Niyaz, M. Laskey, R. Doan, X. Liu, J. Aparicio, and K. Goldberg, "Dex-net 2.0: Deep learning to plan robust grasps with synthetic point clouds and analytic grasp metrics," 07 2017.

[17] A. Mousavian, C. Eppner, and D. Fox, "6-dof graspnet: Variational grasp generation for object manipulation," *CoRR*, vol. abs/1905.10520, 2019. [Online]. Available: http://arxiv.org/abs/1905.10520

[18] G. Neumann, C. Daniel, A. Paraschos, A. Kupcsik, and J. Peters, "Learning modular policies for robotics," *Frontiers in Computational Neuroscience*, vol. 8, p. 62, 2014. [Online]. Available: https://www.frontiersin.org/article/10.3389/fncom.2014.00062

[19] OpenAI, I. Akkaya, M. Andrychowicz, M. Chociej, M. Litwin, B. McGrew, A. Petron, A. Paino, M. Plappert, G. Powell, R. Ribas, J. Schneider, N. Tezak, J. Tworek, P. Welinder, L. Weng, Q. Yuan, W. Zaremba, and L. Zhang, "Solving rubik's cube with a robot hand," 2019.

[20] OpenAI, M. Andrychowicz, B. Baker, M. Chociej, R. Józefowicz, B. McGrew, J. W. Pachocki, J. Pachocki, A. Petron, M. Plappert, G. Powell, A. Ray, J. Schneider, S. Sidor, J. Tobin, P. Welinder, L. Weng, and W. Zaremba, "Learning dexterous in-hand manipulation," *CoRR*, vol. abs/1808.00177, 2018. [Online]. Available: http://arxiv.org/abs/1808.00177

[21] L. Pinto, M. Andrychowicz, P. Welinder, W. Zaremba, and P. Abbeel, "Asymmetric actor critic for image-based robot learning," *CoRR*, vol. abs/1710.06542, 2017. [Online]. Available: http://arxiv.org/abs/1710.06542

[22] L. Pinto and A. Gupta, "Supersizing self-supervision: Learning to grasp from 50k tries and 700 robot hours," *CoRR*, vol. abs/1509.06825, 2015. [Online]. Available: http://arxiv.org/abs/1509.06825

[23] Z. Qin, K. Fang, Y. Zhu, L. Fei-Fei, and S. Savarese, "Keto: Learning keypoint representations for tool manipulation," *arXiv preprint arXiv:1910.11977*, 2019.

[24] A. Rajeswaran, V. Kumar, A. Gupta, J. Schulman, E. Todorov, and S. Levine, "Learning complex dexterous manipulation with deep reinforcement learning and demonstrations," *CoRR*, vol. abs/1709.10087, 2017. [Online]. Available: http://arxiv.org/abs/1709.10087

[25] N. Ratliff, J. A. Bagnell, and S. S. Srinivasa, "Imitation learning for locomotion and manipulation," in *2007 7th IEEE-RAS International Conference on Humanoid Robots*, 2007, pp. 392–397.

[26] M. A. Riedmiller, R. Hafner, T. Lampe, M. Neunert, J. Degrave, T. V. de Wiele, V. Mnih, N. Heess, and J. T. Springenberg, "Learning by playing - solving sparse reward tasks from scratch," *CoRR*, vol. abs/1802.10567, 2018. [Online]. Available: http://arxiv.org/abs/1802.10567

[27] S. Ross, G. J. Gordon, and J. A. Bagnell, "No-regret reductions for imitation learning and structured prediction," *CoRR*, vol. abs/1011.0686, 2010. [Online]. Available: http://arxiv.org/abs/1011.0686

[28] E. Rückert and A. D'Avella, "Learned parametrized dynamic movement primitives with shared synergies for controlling robotic and musculoskeletal systems," *Frontiers in Computational Neuroscience*, vol. 7, p. 138, 2013. [Online]. Available: https://www.frontiersin.org/article/10.3389/fncom.2013.00138

[29] S. Schaal, *Dynamic Movement Primitives -A Framework for Motor Control in Humans and Humanoid Robotics*. Tokyo: Springer Tokyo, 2006, pp. 261–280. [Online]. Available: https://doi.org/10.1007/4-431-31381-8_23

[30] M. Sieb, X. Zhou, A. Huang, O. Kroemer, and K. Fragkiadaki, "Graph-structured visual imitation," in *Conference on Robot Learning (CoRL)*, 2019.

[31] B. D. Silva, G. Konidaris, and A. Barto, "Learning parameterized skills," 2012.

[32] J. T. Springenberg, K. Hausman, M. Riedmiller, N. Heess, and Z. Wang, "Learning an embedding space for transferable robot skills," 2018.

[33] R. Strudel, A. Pashevich, I. Kalevatykh, I. Laptev, J. Sivic, and C. Schmid, "Learning to combine primitive skills: A step towards versatile robotic manipulation," 2019.

[34] F. Stulp, E. Theodorou, and S. Schaal, "Reinforcement learning with sequences of motion primitives for robust manipulation," *Robotics, IEEE Transactions on*, vol. 28, pp. 1360–1370, 12 2012.

[35] E. Todorov, T. Erez, and Y. Tassa, "Mujoco: A physics engine for model-based control." in *IROS*. IEEE, 2012, pp. 5026–5033. [Online]. Available: http://dblp.uni-trier.de/db/conf/iros/iros2012.htmlTodorovET12

[36] T. Tosun, E. Mitchell, B. Eisner, J. Huh, B. Lee, D. Lee, V. Isler, H. S. Seung, and D. D. Lee, "Pixels to plans: Learning non-prehensile manipulation by imitating a planner," *CoRR*, vol. abs/1904.03260, 2019. [Online]. Available: http://arxiv.org/abs/1904.03260

[37] H.-Y. F. Tung, R. Cheng, and K. Fragkiadaki, "Learning spatial common sense with geometry-aware recurrent networks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2019.

[38] N. Wahlström, T. B. Schön, and M. P. Deisenroth, "From pixels to torques: Policy learning with deep dynamical models," *CoRR*, vol. abs/1502.02251, 2015. [Online]. Available: http://arxiv.org/abs/1502.02251

[39] T. Zhang, Z. McCarthy, O. Jow, D. Lee, K. Goldberg, and P. Abbeel, "Deep imitation learning for complex manipulation tasks from virtual reality teleoperation," *2018 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 1–8, 2018.